Architecting Human-Like Memory Decay in Temporal Knowledge Graphs for Multi-Agent Social Simulations

The pursuit of developing autonomous, generative agents capable of sustaining long-term, multi-agent social simulations necessitates a departure from traditional "perfect memory" architectures. In standard Retrieval-Augmented Generation (RAG) paradigms, information is accumulated monotonically in flat vector stores, leading to context bloat, computational inefficiency, and unnatural behavioral plateaus. Human cognition, by contrast, relies on forgetting as a primary feature of intelligence—a mechanism that curates relevance, extracts abstract semantic meaning from granular episodic noise, and optimizes retrieval under strict biological constraints.

Translating the neurobiology of human memory decay into a computational framework requires a sophisticated temporal knowledge graph (TKG) architecture. Using Neo4j and the Graphiti framework, agents can model the dynamic lifecycle of a fact, maintaining both episodic provenance (the raw observation) and semantic entities (the abstracted concept) across temporal validity windows (e.g., valid_at and invalid_at)[1].

The following report systematically analyzes the psychological and neurobiological mechanisms of memory retention and decay, translating each into implementable mathematical formulations, graph property mappings, and explicit Neo4j Cypher operational logic.

1. Emotional Salience and Memory Modulation

Plain-Language Summary of the Finding

Emotional salience drastically alters the encoding and retention trajectory of memories. Neurobiologically, this modulation is driven by the amygdala, which projects to the hippocampus to enhance the consolidation of memory traces during and immediately after emotionally arousing events[4]. Psychological research distinguishes between two independent dimensions of emotion: *arousal* (physiological intensity, ranging from calm to highly excited) and *valence* (subjective quality, ranging from deeply negative to highly positive)[4].

The strength of evidence for arousal-mediated memory enhancement is strong. Easterbrook's cue utilization hypothesis (1959) outlines the "narrowing of attention" phenomenon: under high emotional arousal, an organism experiences a restriction in cue utilization, forcing intense focus on central, critical details while peripheral information is poorly encoded and rapidly forgotten[5]. Regarding valence, evidence suggests a phenomenon known as the Fading Affect Bias (FAB). While both positive and negative memories are retained better than neutral ones, the negative affect associated with unpleasant memories fades significantly faster than the positive affect of pleasant memories, serving as a psychological immune mechanism[8].

Mathematical Formula for Decay

To computationally model emotional salience, the baseline exponential memory decay must be inversely proportional to arousal, adjusted for the Fading Affect Bias, and penalized by attentional narrowing for peripheral details.

Let the base activation $A_i(t)$ of a memory edge i at time t follow an exponential decay function:

$$A_i(t) = A_0 \cdot e^{-d_{mod} \cdot t}$$

The modified decay rate d_{mod} is defined as:

$$d_{mod} = d_{base} \cdot e^{-\kappa \cdot \text{Arousal}_i} \cdot \Phi(\text{Valence}_i)$$

Where:

- $\text{Arousal}_i \in [0, 1]$ represents the physiological intensity of the event.
- $\text{Valence}_i \in [-1, 1]$ represents the emotional tone.
- d_{base} is the baseline decay rate for neutral memories (e.g., 0.05).
- κ is the arousal retention multiplier (Sensible default: 2.0). High arousal exponentially shrinks the decay rate.
- $\Phi(\text{Valence}_i)$ represents the Fading Affect Bias modifier. If $\text{Valence}_i < 0$ (negative), $\Phi = 1.2$ (accelerating the decay of the negative edge slightly over time compared to positive). If $\text{Valence}_i > 0$, $\Phi = 0.9$.

To implement Easterbrook's attentional narrowing, peripheral details (determined by semantic distance or graph distance from the central event entity) receive a reduced initial encoding weight A_0:

$$A_0 = 1.0 - (\text{Arousal}_i \cdot \delta \cdot \text{Distance})$$

Where δ (default 0.3) is the peripheral blinding factor.

Graph Storage Mapping

In Graphiti, events are ingested as Episode nodes, which then generate Entity nodes connected by RELATES_TO edges[1].

- **Properties to add:** The Episode node receives arousal (Float) and valence (Float). The resulting RELATES_TO edges inherit these properties, along with a centrality_score (Float) indicating how peripheral the fact was to the emotional core.
- **Update Mechanism:** At ingestion, an LLM evaluates the raw episode text to score arousal and valence, propagating these to the edges.

Implementation Logic (Cypher)

(a) Scheduled Decay Job:

```
// Nightly job to decay the activation weight of emotional memories
MATCH (ep:Episode)-[r:RELATES_TO]->(entity:Entity)
WHERE r.invalid_at IS NULL
WITH r, duration.inDays(r.valid_at, datetime()).days AS days_elapsed
// Apply Fading Affect Bias (FAB) multiplier
WITH r, days_elapsed,
    CASE WHEN coalesce(r.valence, 0.0) < 0 THEN 1.2
        WHEN coalesce(r.valence, 0.0) > 0 THEN 0.9
        ELSE 1.0 END AS fab_modifier
// Calculate modified decay rate
WITH r, days_elapsed,
    0.05 * exp(-2.0 * coalesce(r.arousal, 0.0)) * fab_modifier AS d_mod
// Update the current activation weight on the edge
SET r.current_weight = coalesce(r.initial_weight, 1.0) * exp(-d_mod * days_elapsed)
```

(b) Retrieval-Time Scoring:

```
Cypher
// Combine vector similarity with emotionally modulated weight
CALL db.index.vector.queryNodes('entity_embeddings', 20, $query_embedding) YIELD
node AS entity, score AS semantic_sim
MATCH (agent:Agent {id:
$agent_id})-[:EXPERIENCED]->(ep:Episode)-[r:RELATES_TO]->(entity)
WHERE r.invalid_at IS NULL
// Score combines semantic relevance with the emotionally modulated current_weight
WITH entity, r, ep, (semantic_sim * 0.7) + (coalesce(r.current_weight, 0.1) * 0.3) AS
retrieval_score
WHERE retrieval_score > 0.4
RETURN entity.name, r.fact, ep.arousal, retrieval_score
ORDER BY retrieval_score DESC LIMIT 10;
```

(c) Pruning Rules:

```
Cypher
// Prune peripheral details of highly arousing events (Easterbrook Hypothesis)
MATCH (ep:Episode)-[r:RELATES_TO]->(entity:Entity)
WHERE r.invalid_at IS NULL AND r.arousal > 0.8 AND r.centrality_score < 0.3
// High arousal + low centrality = peripheral detail that should be forgotten rapidly
SET r.invalid_at = datetime(), r.eviction_reason = "easterbrook_peripheral_decay"
RETURN count(r) AS pruned_peripheral_facts;
```

2. Flashbulb and Highly Significant Memories

Plain-Language Summary of the Finding

Flashbulb memories are highly detailed, exceptionally vivid snapshot memories of the moment and circumstances in which a piece of surprising and consequential (or emotionally arousing) news was learned. Historically, these memories were theorized to be "printed" permanently in the brain, immune to forgetting. However, contemporary longitudinal psychological research demonstrates that the objective, factual details of flashbulb memories decay at rates highly similar to everyday episodic memories[11]. What differentiates them is not objective persistence, but the individual's subjective *confidence* in their accuracy and their phenomenal *vividness*. The evidence is strong that agents should not retain perfect factual recall of flashbulb events; rather, they should retain the "gist" coupled with an unshakeable, often hallucinatory, confidence in the memory's fidelity.

Mathematical Formula for Decay

To model flashbulb phenomena, the agent architecture must decouple the objective memory strength (Factual Activation, $A_i(t)$) from subjective meta-cognitive certainty (Confidence, $C_i(t)$).

$$A_i(t) = A_0 \cdot e^{-d_{mod} \cdot t}$$

$$C_i(t) = \text{Arousal}_i + (1 - \text{Arousal}_i) \cdot e^{-\lambda_{conf} \cdot t}$$

Where:

- $A_i(t)$ defines the probability of correctly returning the exact properties of the edge to the LLM prompt.
- $C_i(t)$ defines the agent's subjective confidence (0 to 1).
- λ_{conf} is the normal confidence decay rate. For flashbulb memories (

$\text{Arousal} > 0.85$), the first term heavily anchors $C_i(t)$ near 1.0, rendering confidence virtually immune to time.

Graph Storage Mapping

- **Properties to add:** confidence (Float) on the RELATES_TO edge, and a boolean flag is_flashbulb triggered at ingestion when $\text{Arousal} > 0.85$.
- **Update Mechanism:** The scheduled decay job applies standard temporal decay to the fact's factual weight but leaves the confidence score artificially elevated for flashbulb events.

Implementation Logic (Cypher)

(a) Scheduled Decay Job:

```cypher
MATCH (ep:Episode)-[r:RELATES_TO]->(entity:Entity)
WHERE r.invalid_at IS NULL AND r.is_flashbulb = TRUE
WITH r, duration.inDays(r.valid_at, datetime()).days AS days_elapsed
// Factual weight decays normally
SET r.current_weight = exp(-0.05 * days_elapsed),
// Confidence remains artificially high based on the formula
   r.confidence = r.arousal + ((1.0 - r.arousal) * exp(-0.01 * days_elapsed))
```

(b) Retrieval-Time Scoring (Pseudocode for Context Assembly):

```python
# The retrieval query fetches both current_weight and confidence
query = """
MATCH (ep:Episode)-[r:RELATES_TO]->(entity:Entity)
WHERE r.invalid_at IS NULL AND r.confidence > 0.5
RETURN r.fact, r.current_weight, r.confidence
ORDER BY r.confidence DESC LIMIT 5
"""
memories = graph.run_cypher(query)
prompt_context = "Agent Memory Context:\n"

for mem in memories:
   if mem.current_weight < 0.3 and mem.confidence > 0.8
```

```
        # High confidence, low factual accuracy (Flashbulb distortion)
        prompt_context += f"- You vaguely recall {mem.fact}, but you are ABSOLUTELY CERTAIN about it, and will aggressively defend this memory even if you are missing specific details.\n"
    elif mem.current_weight > 0.7:
        # Strong factual memory
        prompt_context += f"- You clearly recall: {mem.fact}.\n"
```

(c) Pruning Rules:

```
Cypher
// Flashbulb memories are never truly deleted, but their specific granular details are summarized
MATCH (ep:Episode)-[r:RELATES_TO]->(entity:Entity)
WHERE r.invalid_at IS NULL AND r.is_flashbulb = TRUE AND r.current_weight < 0.1
// Instead of deleting, we flag it for LLM hallucination/gist generation in the next cycle
SET r.requires_reconstruction = TRUE
RETURN count(r) AS flashbulb_memories_awaiting_reconstruction;
```

3. Reinforcement and Consolidation

Plain-Language Summary of the Finding

Every time a memory is retrieved, it is not merely read like a static file; it is biologically reconstructed and reconsolidated. The Testing Effect and Spaced Repetition paradigms prove that actively retrieving a memory halts its current decay trajectory and significantly flattens the future forgetting curve. The evidence for this is universally strong and forms the foundation of cognitive architectures like ACT-R[12] and modern spaced-repetition algorithms like the Free Spaced Repetition Scheduler (FSRS)[15]. Conversely, reconsolidation theory posits that the act of retrieval renders the memory temporarily labile, making it vulnerable to alteration or updating.

Mathematical Formula for Decay

The ACT-R base-level activation equation models the logarithmic summation of individual retrieval instances over time[13]:

$$B_i = \ln\left(\sum_{k=1}^{n}(t_{current} - t_k)^{-d}\right)$$

Where n is the total access count, t_k are the specific timestamps of previous accesses, and d is the standard decay parameter (default 0.5).

While highly accurate, storing an unbounded array of historical timestamps t_k per edge in a graph database is computationally prohibitive. Therefore, implementing an iterative FSRS-inspired stability approximation is vastly superior[15]:

$$R(t) = 9^{\frac{-t}{S}}$$

Where $R(t)$ is Retrievability (0 to 1), t is time since last access, and S is Memory Stability (the time in days it takes for R to drop to 90%).

Upon successful retrieval by the agent, Stability S increases recursively:

$$S_{new} = S_{old} \cdot \left(1 + e^{\omega \cdot (1 - R(t))}\right)$$

- ω: Reinforcement scalar (default 2.5). If the memory was almost forgotten (low $R(t)$), successfully retrieving it yields a massive boost to future stability, reflecting the Testing Effect.

Graph Storage Mapping

- **Properties to add:** stability (Float, representing S), last_accessed (Datetime), access_count (Integer).
- **Update Mechanism:** These properties reside on the RELATES_TO edge. Every time an agent queries the graph and utilizes a specific edge in its LLM context window (i.e., a successful retrieval), an asynchronous job updates the edge to reconsolidate it.

Implementation Logic (Cypher)

(a) Scheduled Decay Job:

Cypher

```
// Update Retrievability R(t) based on the FSRS approximation
MATCH ()-[r:RELATES_TO]->()
WHERE r.invalid_at IS NULL AND r.stability IS NOT NULL
WITH r, duration.inDays(r.last_accessed, datetime()).days AS t
WITH r, 9^(-(toDbMath(t) / r.stability)) AS current_retrievability
SET r.retrievability = current_retrievability
```

(b) Retrieval-Time Scoring:

```
Cypher
// After the agent successfully uses a set of memories, trigger a reconsolidation update
UNWIND $retrieved_edge_ids AS edge_id
MATCH ()-[r:RELATES_TO]->() WHERE elementId(r) = edge_id
WITH r, duration.inDays(r.last_accessed, datetime()).days AS t, coalesce(r.stability, 1.0) AS S
WITH r, t, S, 9^(-(toDbMath(t) / S)) AS R
// Calculate new stability based on spacing effect multiplier (omega = 2.5)
WITH r, S * (1 + exp(2.5 * (1 - R))) AS new_S
// Update the edge: reconsolidation successful
SET r.last_accessed = datetime(),
    r.access_count = coalesce(r.access_count, 0) + 1,
    r.stability = new_S
```

(c) Pruning Rules:

```
Cypher
// Remove edges that have failed to be reinforced and have fallen below the retrievability threshold
MATCH ()-[r:RELATES_TO]->()
WHERE r.invalid_at IS NULL AND r.retrievability < 0.05 AND coalesce(r.access_count, 0) < 2
// Soft delete
SET r.invalid_at = datetime(), r.eviction_reason = "failed_consolidation"
RETURN count(r) AS forgotten_unreinforced_memories;
```

4. Memory Types: Episodic to Semantic Transition

Plain-Language Summary of the Finding

Human memory relies on two distinct declarative systems: *Episodic Memory* (autobiographical, rich in contextual and temporal details) and *Semantic Memory* (general knowledge, devoid of the specific context in which it was learned). According to the Memory Chain Model (Murre & Chessa), memory consolidation is a continuous multi-stage process where rapidly decaying traces in the hippocampus (episodic) are gradually transferred to the neocortex, where they decay at near-zero rates (semantic)[19]. Similarly, Fuzzy Trace Theory suggests humans encode both a detailed "verbatim" trace and a meaning-based "gist" trace; the verbatim trace vanishes quickly, leaving only the semantic gist[22]. Evidence for this dual-process transition is strong.

Mathematical Formula for Decay

The Memory Chain Model expresses the probability of recall as the sum of intensities of the fast hippocampal (r_1) and slow neocortical (r_2) processes[20]:

$$p_{recall}(t) = 1 - \exp\left(-(r_1(t) + r_2(t))\right)$$

Where intensities decay as:

$$r_1(t) = \mu_1 \cdot e^{-a_1 t}$$

$$r_2(t) = \mu_1 \mu_2 \frac{1 - e^{-a_1 t}}{a_1} \cdot e^{-a_2 t}$$

- a_1 : Fast decay rate of episodic details (e.g., 0.1 per day).
- a_2 : Slow decay rate of semantic gist (e.g., 0.001 per day).
- μ_1 : Initial encoding intensity.
- μ_2 : Rate of consolidation transfer from episodic to semantic.

Graph Storage Mapping

Graphiti intrinsically supports this via its Episode nodes (representing the verbatim transcript/interaction timeline) and Entity nodes (the generalized semantic knowledge)[1].

- **Properties to add:** memory_type (String: 'episodic' or 'semantic') on edges.
- **Update Mechanism:** As multiple 'episodic' edges connecting the same entities age, they trigger an LLM summarization step. The LLM extracts the "gist" and writes a new 'semantic' edge with a much lower baseline decay rate (a_2), subsequently invalidating the old episodic edges (transitioning from state r_1 to r_2).

Implementation Logic (Pseudocode and Cypher)

(a) Scheduled Decay Job:

Cypher

```
// Episodic and Semantic edges decay at vastly different rates
MATCH ()-[r:RELATES_TO]->()
WHERE r.invalid_at IS NULL
WITH r, duration.inDays(r.valid_at, datetime()).days AS days_elapsed
SET r.current_weight = CASE
  WHEN r.memory_type = 'episodic' THEN exp(-0.1 * days_elapsed)
```

```
    WHEN r.memory_type = 'semantic' THEN exp(-0.001 * days_elapsed)
    END
```

(b) Retrieval-Time Scoring:

```cypher
// Semantic memories have inherently higher utility for general reasoning tasks
MATCH (ep:Episode)-[r:RELATES_TO]->(entity:Entity)
WHERE r.invalid_at IS NULL
WITH entity, r, ep,
    CASE WHEN r.memory_type = 'semantic' THEN 1.5 ELSE 1.0 END AS type_multiplier
WITH entity, r, ep, (r.current_weight * type_multiplier) AS final_score
WHERE final_score > 0.3
RETURN entity.name, r.fact, r.memory_type, final_score
ORDER BY final_score DESC LIMIT 15;
```

(c) Pruning Rules (Consolidation Pipeline):

```python
# 1. Identify highly activated episodic clusters ready for semantic consolidation
query = """
MATCH (ep:Episode)-[r:RELATES_TO]->(entity:Entity)
WHERE r.invalid_at IS NULL AND r.memory_type = 'episodic' AND r.access_count > 3
RETURN entity.id, entity.name, collect(r.fact) as episodic_facts
"""
clusters = graph.run_cypher(query)

# 2. LLM Processing: Extract semantic gist (Fuzzy Trace Theory)
for cluster in clusters:
   prompt = f"Distill these specific episodic interactions into a timeless, general semantic fact about {cluster['entity.name']}: {cluster['episodic_facts']}"
   gist = llm.generate(prompt)

   # 3. Cypher: Create semantic edge and invalidate old episodic edges
   update_query = """
   MATCH (entity:Entity {id: $entity_id})
   // Create new semantic edge
```

```
MERGE (agent:Agent)-[new_r:RELATES_TO {memory_type: 'semantic'}]->(entity)
SET new_r.fact = $gist, new_r.valid_at = datetime(), new_r.stability = 100.0

// Soft-delete the old verbatim data
WITH entity
MATCH (ep:Episode)-[old_r:RELATES_TO]->(entity)
WHERE old_r.invalid_at IS NULL AND old_r.memory_type = 'episodic'
SET old_r.invalid_at = datetime(), old_r.eviction_reason = "consolidated_to_semantic"
"""
graph.run_cypher(update_query, entity_id=cluster['entity.id'], gist=gist)
```

5. Interference and Relevance

Plain-Language Summary of the Finding

Time is not the sole, or even primary, driver of forgetting. As proposed by Underwood (1957) and confirmed by decades of cue-overload research, forgetting is largely driven by *interference*—both proactive (old memories blocking new ones) and retroactive (new memories overwriting old ones)[24]. When an agent learns 50 distinct facts about an entity named "Kendra," retrieving a specific early fact ("Kendra loves Adidas shoes") becomes exceedingly difficult because the retrieval cue ("Kendra") is overloaded. This is formalized in the ACT-R cognitive architecture as the "fan effect." Evidence for interference is exceptionally strong.

Mathematical Formula for Decay

To incorporate interference, the final Activation A_{final} of a memory node is penalized by the sheer volume of competing associations (its degree centrality in the graph).

$$A_{final} = A_{temporal} - \gamma \cdot \ln(\text{Degree}(node))$$

Where:

- $A_{temporal}$ is the current base activation after normal time decay.
- $\text{Degree}(node)$ is the total number of active edges connected to the cue entity.
- γ is the interference penalty scalar (default 0.15).

To offset this penalty, *relevance*—computed via vector cosine similarity between the current cognitive context and the memory's semantic embedding—acts as a targeted multiplier, cutting through the noise.

$$\text{Score} = A_{final} + (\alpha \cdot \text{CosineSimilarity})$$

Graph Storage Mapping

- **Properties to add:** None strictly required for degree calculation, as Graphiti and Neo4j

intrinsically support $O(1)$ relationship counting. Semantic embeddings of the edge properties must be maintained in a vector index to compute relevance dynamically[1].

Implementation Logic (Cypher)

(a) Scheduled Decay Job:

Cypher

```
// Interference does not technically decay the physical trace in biological systems,
// rather it hinders retrieval. Therefore, no physical decay job is required for interference.
// The penalty is calculated purely at retrieval time.
```

(b) Retrieval-Time Scoring (Fan Effect + Relevance):

Cypher

```
// 1. Retrieve candidates via Vector Index (Cosine Similarity / Relevance)
CALL db.index.vector.queryNodes('entity_embeddings', 20, $query_embedding) YIELD
node AS entity, score AS semantic_sim
MATCH (agent:Agent)-[r:RELATES_TO]->(entity)
WHERE r.invalid_at IS NULL

// 2. Retrieve Temporal Activation (using FSRS approximation)
WITH entity, r, semantic_sim, duration.inDays(coalesce(r.last_accessed, r.valid_at),
datetime()).days AS t
WITH entity, r, semantic_sim, 9^(-(toDbMath(t) / coalesce(r.stability, 1.0))) AS a_temporal

// 3. Calculate Cue Overload (Fan Effect / Interference)
// SIZE() provides a highly optimized degree lookup in Neo4j
WITH entity, r, semantic_sim, a_temporal, SIZE((entity)-[:RELATES_TO]-()) AS node_degree
WITH entity, r, semantic_sim, a_temporal,
    a_temporal - (0.15 * log(node_degree + 1)) AS a_final

// 4. Final Score Calculation (Interference vs. Relevance)
WITH entity, r, a_final + (2.0 * semantic_sim) AS final_score
WHERE final_score > 0.4 // Threshold for conscious retrieval
RETURN entity.name, r.fact, final_score
ORDER BY final_score DESC
```

(c) Pruning Rules:

```
// Retroactive interference pruning: If an entity has too many edges (high fan),
// prune the oldest, least accessed edges to preserve cognitive clarity.
MATCH (entity:Entity)
WHERE SIZE((entity)-[:RELATES_TO]-()) > 50
WITH entity
MATCH (entity)-[r:RELATES_TO]-()
WHERE r.invalid_at IS NULL
WITH r ORDER BY coalesce(r.access_count, 0) ASC, r.valid_at ASC LIMIT 10
SET r.invalid_at = datetime(), r.eviction_reason = "retroactive_interference_overload"
RETURN count(r) AS pruned_due_to_overload;
```

6. Computational Models of Agentic Memory

Plain-Language Summary of the Finding

Recent advances in LLM-agent memory frameworks have formulated explicit algorithms for managing finite context limits. Generative Agents (Park et al.) utilize a tripartite heuristic: Recency + Importance + Relevance[26]. A-MEM utilizes a Zettelkasten-inspired continuous evolution process, merging and dynamically updating notes via autonomous LLM reflection[27]. MemoryOS implements hierarchical memory routing, shifting short-term FIFO (First-In, First-Out) context into a heat-based long-term store[30].

The most rigorous mathematical formalization of these constraints is found in the "Forgetful but Faithful" Memory-Aware Retention Schema (MaRS) (ArXiv 2512.12856)[31]. MaRS frames agent memory as a strict "bounded budget" problem. Rather than infinite accumulation, it treats retention as a submodular knapsack problem, where agents must purposefully forget low-utility data while preserving "faithful" provenance and adhering to privacy and computational boundaries. Evidence for the effectiveness of budgeted computational models over infinite-context retrieval is strong, vastly saving token costs and reducing LLM hallucination[33].

Model Framework	**Core Mechanism**	**Primary Metric**
Generative Agents [cite: 26]	Heuristic Scoring	Score = α (Recency) + β (Importance) + γ

		(Relevance)
A-MEM [cite: 27, 29]	Zettelkasten Evolution	LLM-driven organic linking and context evolution
MemoryOS [cite: 30]	Hierarchical Routing	Short-term FIFO $\rightarrow$ Mid-term $\rightarrow$ Long-term heat-based storage
MaRS [cite: 31]	Bounded Budget / Knapsack	Density Score = (Utility - Privacy) / Token Weight

Mathematical Formula for Decay

MaRS unifies disparate variables into a single Density Score for eviction[31]:

$$\text{Score}_i = \frac{\hat{U}_i - \lambda_{priv} \cdot s_i}{w_i}$$

Where:

- $\hat{U}_i$: Unpenalized utility proxy (a vector combining exponential age decay, access frequency, semantic similarity, and graph centrality).
- $s_i \in [0, 1]$: Privacy sensitivity score (e.g., PII or highly sensitive social data).
- w_i : Computational weight (token footprint of the memory node).
- λ_{priv} : Privacy/eviction regularization parameter.

If the agent's total memory token count exceeds a predefined budget B , items with the lowest Score_i are evicted or summarized.

Graph Storage Mapping

- **Properties to add:** token_weight (Integer), privacy_sensitivity (Float), importance (Float).
- **Update Mechanism:** MaRS-style pruning occurs asynchronously. A scheduled job evaluates the total token_weight of all active edges. If the budget constraint is breached, the job identifies targets for invalidation based on the Density Score.

Implementation Logic (Cypher)

(a) Scheduled Decay Job (Importance/Budget Tracking):

Cypher

```
// Update baseline importance over time (Generative Agents decay style)
MATCH ()-[r:RELATES_TO]->()
WHERE r.invalid_at IS NULL
WITH r, duration.inDays(r.valid_at, datetime()).days AS days_elapsed
SET r.decayed_importance = coalesce(r.importance, 1.0) * exp(-0.02 * days_elapsed)
```

(b) Retrieval-Time Scoring:

Cypher

```
// Retrieve nodes utilizing the MaRS unpenalized utility proxy equation
MATCH (agent:Agent)-[r:RELATES_TO]->(entity:Entity)
WHERE r.invalid_at IS NULL
// Utility combines Recency (FSRS), Importance, and Centrality
WITH r, entity,
    (9^(-(duration.inDays(r.valid_at, datetime()).days / coalesce(r.stability, 1.0)))) AS recency,
    coalesce(r.decayed_importance, 1.0) AS importance,
    SIZE((entity)-[:RELATES_TO]-()) AS centrality
WITH r, entity, (recency * 0.4) + (importance * 0.4) + (centrality * 0.2) AS utility
RETURN entity.name, r.fact, utility
ORDER BY utility DESC LIMIT 10
```

(c) Pruning Rules for Bounded-Budget Eviction (MaRS Implementation):

Cypher

```
// Step 1: Calculate total budget consumed by active memories
MATCH ()-[r:RELATES_TO]->()
WHERE r.invalid_at IS NULL
WITH sum(coalesce(r.token_weight, 15)) AS total_tokens

// Step 2: Trigger pruning if budget exceeded (e.g., 50,000 tokens)
CALL apoc.when(
   total_tokens > 50000,
```

```
    '
    MATCH ()-[r:RELATES_TO]->() WHERE r.invalid_at IS NULL
    // Calculate Utility
    WITH r, duration.inDays(coalesce(r.last_accessed, r.valid_at), datetime()).days AS t
    WITH r, t, coalesce(r.importance, 1.0) AS importance,
        coalesce(r.privacy_sensitivity, 0.0) AS privacy,
        coalesce(r.token_weight, 15) AS weight

    WITH r, (9^(-(toDbMath(t) / coalesce(r.stability, 1.0))) * importance) AS utility, privacy,
weight
    // MaRS Density Score Calculation
    WITH r, (utility - (0.5 * privacy)) / weight AS density_score
    ORDER BY density_score ASC
    // Select bottom 5% for eviction
    LIMIT 100

    // Soft-delete to preserve provenance
    SET r.invalid_at = datetime(),
       r.eviction_reason = "mars_budget_pruning"
    RETURN count(r) AS evicted_count
    ',
    'RETURN 0 AS evicted_count',
    {total_tokens: total_tokens}
) YIELD value
RETURN value.evicted_count;
```

7. Recommended Minimal Design

Synthesizing human neurobiology, psychological cognitive models, and computational optimizations yields the following minimal, highly implementable Graphiti/Neo4j schema. This setup captures the vast majority of human-like forgetting benefits without crippling the system with computational overhead.

Minimal Graph Schema

Instead of managing dozens of independent decay modifiers, standardizing on a recursive, FSRS-inspired stability model mixed with the Generative Agents' static importance score offers the best performance-to-fidelity ratio.

Element	Type	Core Properties
Agent	Node	id, name, token_budget
Episode	Node	transcript, arousal (0-1),

		valence (-1 to 1)
Entity	Node	name, type, embedding (Vector representation)
RELATES_TO	Edge	fact, valid_at, invalid_at, last_accessed, stability, access_count, memory_type

The Unified Retrieval Formula

At query time, rank candidate edges using a fusion of Semantic Similarity, Temporal Retrievability, and Interference:

$$\text{Rank}_i = (W_{sim} \cdot \text{Cosine}(i, q)) + \left(W_{ret} \cdot 9^{\frac{-t_i}{S_i}}\right) - (W_{int} \cdot \ln(\deg_i))$$

Where: t_i is time since last_accessed, S_i is the mathematically calculated stability (initialized via arousal and importance scores), and $\deg_i$ is the node degree (calculating interference).

The Consolidation Loop

A scheduled daily job evaluates all episodic edges. If access_count > 3 or stability > X, the system passes the associated facts to an LLM to generate a summarized semantic edge attached directly to the Entity, setting the old episodic edge's invalid_at property to the current datetime. This satisfies the Fuzzy Trace Theory, limits token context bloat, and mirrors hippocampal-neocortical transfer natively within Neo4j.

8. Open Problems in Temporal Agentic Memory

While the proposed architecture successfully models healthy human memory decay and retention, several open challenges remain at the frontier of social simulations:

1. **Reconsolidation Corruption (The Misinformation Effect):** In human cognition, the act of retrieving a memory places it in a labile state, making it highly susceptible to corruption by new, false context (the misinformation effect). Developing computational mechanisms where LLM agents dynamically *overwrite* or morph the fact string based on biased retrieval contexts—rather than merely creating parallel contradictory edges—remains mathematically unformalized in strict graph databases.
2. **Dynamic Ontology Drift:** As episodic nodes are abstracted into semantic gist, the agent's internal ontology (e.g., custom entity types in Graphiti) must adapt[2]. Creating unsupervised algorithms that allow the graph to spawn new relationship labels autonomously (e.g., slowly evolving a WORKS_WITH edge into a TRUSTS edge based on

the accumulation of positive emotional valence) without breaking database schema constraints is an active area of research.

3. **Cross-Agent Contagion:** Current frameworks like MaRS and A-MEM focus almost exclusively on single-agent privacy and utility[27]. However, multi-agent social simulations require mechanisms for *memory contagion*—how the emotional valence of a shared event propagates heterogeneously across multiple agents' graphs, with each agent experiencing different Easterbrook attentional narrowing effects depending on their spatial or social proximity to the event[11]. Implementing this requires complex multi-modal graph traversal algorithms that currently strain standard vector-RAG pipelines.

References

1. Alqithami, S. (2025). Forgetful but Faithful: A Cognitive Memory Architecture and Benchmark for Privacy-Aware Generative Agents. *arXiv preprint arXiv:2512.12856*.
2. Anderson, J. R., Bothell, D., Byrne, M. D., Douglass, S., Lebiere, C., & Qin, Y. (2004). An integrated theory of the mind. *Psychological Review, 111*(4), 1036-1060.
3. Easterbrook, J. A. (1959). The effect of emotion on cue utilization and the organization of behavior. *Psychological Review, 66*(3), 183-201.
4. Li, W., Nie, D., et al. (2026). Dual-Layer Agentic Memory with Fast Write Routing and Slow Consolidation. *arXiv preprint arXiv:2608.22215*.
5. Murre, J. M. J., & Chessa, A. G. (2011). A Mathematical Model of Forgetting and Amnesia. *Frontiers in Psychology*.
6. Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*.
7. Reyna, V. F., & Brainerd, C. J. (1995). Fuzzy-trace theory: An interim synthesis. *Learning and Individual Differences, 7*(1), 1-75.
8. Talarico, J. M., & Rubin, D. C. (2003). Confidence, not consistency, characterizes flashbulb memories. *Psychological Science, 14*(5), 455-461.
9. Underwood, B. J. (1957). Interference and forgetting. *Psychological Review, 64*(1), 49-60.
10. Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., & Zhang, Y. (2025). A-MEM: Agentic Memory for LLM Agents. *arXiv preprint arXiv:2502.12110*.

Works cited

1. graphiti-core - PyPI, https://pypi.org/project/graphiti-core/0.8.0/
2. getzep/graphiti: Build Real-Time Knowledge Graphs for AI Agents, https://github.com/getzep/graphiti
3. Graphiti Review: Temporal Knowledge Graphs for AI Agents, https://hysenlabs.com/projects/getzep-graphiti
4. How Emotional Arousal Enhances Episodic Memory, https://pure.manchester.ac.uk/ws/files/59231223/Learning_and_Memory_02019_How_emotional_arousal_enhances_episodic_memory.pdf
5. Easterbrook's Hypothesis Definition | Psychology Glossary, https://www.alleydog.com/glossary/definition.php?term=Easterbrook%27s+Hypot

hesis
6. Easterbrook J A. The effect of emotion on cue utilization and the, https://garfield.library.upenn.edu/classics1982/A1982NE45000001.pdf
7. How arousal modulates memory: Disentangling the effects of, https://affectivebrain.com/wp-content/uploads/2014/09/How-Emotional-Arousal-Modulates-Memory-Disentangling-the-Effects-of-Attention-and-Retention.pdf
8. The effect of emotion on cue utilization and the organization of, https://www.researchgate.net/publication/9908785_The_effect_of_emotion_on_cue_utilization_and_the_organization_of_behavior
9. Threat But Not Arousal Narrows Attention: Evidence from Pupil, https://pmc.ncbi.nlm.nih.gov/articles/PMC3204575/
10. Custom Entity and Edge Types - Graphiti - Zep Documentation, https://help.getzep.com/graphiti/core-concepts/custom-entity-and-edge-types
11. Eyewitness memory for arousing events: Putting things into context, https://www.researchgate.net/publication/229775272_Eyewitness_memory_for_arousing_events_Putting_things_into_context
12. The dynamics of cognition: An ACT-R model of cognitive arithmetic, http://act-r.psy.cmu.edu/wordpress/wp-content/uploads/2012/12/459459.pdf
13. Computationally Efficient Approximation of the Base-Level Learning, https://alexpetrov.com/pub/iccm06/PetrovICCM06.pdf
14. ACT-R - Wikipedia, https://en.wikipedia.org/wiki/ACT-R
15. A technical explanation of the FSRS algorithm : r/Anki - Reddit, https://www.reddit.com/r/Anki/comments/18tnp22/a_technical_explanation_of_the_fsrs_algorithm/
16. ABC of FSRS · open-spaced-repetition/awesome-fsrs Wiki - GitHub, https://github.com/open-spaced-repetition/awesome-fsrs/wiki/ABC-of-FSRS
17. FSRS Algorithm in Production: How We Built a Spaced Repetition App, https://teamzlab.com/blog/fsrs-algorithm-spaced-repetition-app
18. Implementing FSRS in 100 Lines - Fernando Borretti, https://borretti.me/article/implementing-fsrs-in-100-lines
19. Mathematical modeling of human memory - Frontiers, https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2023.1298235/full
20. A Mathematical Model of Forgetting and Amnesia - PMC - NIH, https://pmc.ncbi.nlm.nih.gov/articles/PMC3584298/
21. Answering attitudinal questions: modelling the response process, https://research-portal.uu.nl/ws/files/62915012/answering.pdf
22. Meaning, memory, and development in Fuzzy-Trace Theory, https://www.sas.upenn.edu/~baron/journal/11/111031/jdm111031.html
23. (PDF) A Mathematical Model of Forgetting and Amnesia, https://www.researchgate.net/publication/235757229_A_Mathematical_Model_of_Forgetting_and_Amnesia
24. Google Sports Data, https://support.google.com/knowledgepanel/answer/9787176
25. GraphRAG Python package: Accelerating GenAI with knowledge,

https://neo4j.com/blog/news/graphrag-python-package/
26. [Paper reading] Generative Agents: Interactive Simulacra of Human, https://blog.yujieliu.com/blogs/paper-reading/Generative%20Agents_%20Interactive%20Simulacra%20of%20Human%20Behavior.html
27. A-MEM: Agentic Memory for LLM Agents | alphaXiv, https://www.alphaxiv.org/abs/2502.12110v1
28. A-Mem: Agentic Memory for LLM Agents - arXiv, https://arxiv.org/html/2502.12110v2
29. A-MEM: Agentic Memory for LLM Agents - arXiv, https://arxiv.org/pdf/2502.12110
30. Memory in the LLM Era: Modular Architectures and Strategies ... - arXiv, https://arxiv.org/html/2604.01707v2
31. [2512.12856] Forgetful but Faithful: A Cognitive Memory Architecture, https://arxiv.org/abs/2512.12856
32. Forgetful but Faithful: A Cognitive Memory Architecture and ... - arXiv, https://arxiv.org/pdf/2512.12856
33. Forget to Improve: On-Device LLM-Agent Continual Learning ... - arXiv, https://arxiv.org/html/2606.25115v1
34. GRAPH TYPE – Schema enforcement made easy - Neo4j, https://neo4j.com/blog/developer/graph-type-schema-enforcement-made-easy-preview/